

AI observability is the practice of collecting and analyzing telemetry, data such as logs, metrics, and traces that a system automatically records as it runs, from artificial intelligence systems deployed in production, in order to understand how they behave and how well they perform in terms of accuracy, cost, and safety. It applies the idea of observability, which comes from control theory and software engineering, to systems based on machine learning models, and in particular to large language models (LLMs) and autonomous agents. In control theory, observability is a measure of how well the internal state of a system can be inferred from its external outputs.

AI observability is often distinguished from conventional monitoring. Where monitoring relies on predefined metrics, thresholds, and alerts to signal that a problem has occurred, observability aims to provide enough context to understand why it occurred. The distinction matters for AI systems in particular, because a system can appear healthy by standard infrastructure measures, such as latency or error rate, while still producing answers that are wrong, irrelevant, or unsafe.

One of the signals that AI observability tracks is the change in a model's behavior over time, a phenomenon studied in machine learning under the name of concept drift. AI observability is closely related to MLOps (machine learning operations), and it is sometimes treated as one of its parts. When it is applied specifically to language models or to agent-based systems, it is also called LLM observability or agent observability.

Background

The term observability originates in control theory, where it was introduced by Rudolf E. Kálmán in 1960 to describe how well the internal states of a dynamical system can be inferred from measurements of its outputs. In software engineering, the term became popular from the mid-2010s to describe the ability to understand the internal state of complex distributed systems from the telemetry they produce, commonly organized into the "three pillars" of logs,

metrics, and traces.

Applying observability to AI systems raises challenges that do not arise in conventional software. In traditional systems, identical inputs and states are expected to produce identical outputs, and faults can typically be traced to specific code sections. Machine learning models sample from probability distributions instead, so a system may show normal operational characteristics while still generating undesirable outputs such as hallucinations. That gap motivated adding behavioral and semantic telemetry, covering prompt and response content, output quality, and model drift, alongside the conventional signals of software monitoring.

Observability versus monitoring

Although the two terms are sometimes used as synonyms, the literature usually keeps them separate. Monitoring is defined as the collection of predefined measurements against known thresholds, producing alerts when those thresholds are crossed; it addresses questions that operators already know to ask. Observability is the broader capability, allowing operators to investigate unanticipated behavior by correlating diverse signals across the system rather than responding only to known alert conditions. A monitoring-only approach therefore depends on experienced engineers knowing in advance what to look for, while observability supports open-ended investigation of any anomaly.

In the context of AI systems, this distinction is often framed in terms of an additional, behavioral dimension layered on the conventional pillars of telemetry. Infrastructure-level signals may indicate that a model is responding quickly and without errors, while separate signals are required to assess whether its outputs are accurate, relevant, compliant with policy, and free of unsafe content.

Signals and dimensions

AI observability builds on the standard telemetry signals of software engineering, namely

logs, metrics, and traces, and adds dimensions specific to machine learning models. The dimensions that are most often described are:

Inputs and outputs: the prompts sent to a model and the responses it returns. These are sometimes stored in full so that problems can be studied later.

Cost and resource usage: how many tokens a request uses and how much it costs, together with latency. These are common measures of inference efficiency and help identify waste and unexpected expense.

Performance: latency, throughput, and error rate, as in traditional application performance management.

Output quality: how relevant, accurate, and well aligned with the prompt the answer is. These properties usually cannot be checked by normal code, and they include the rate of hallucinations.

Behavior over time: changes in the model, known as drift, and worse results after a model or prompt change.

Safety and policy: finding policy violations, unsafe content, and unusual usage.

For agent-based systems, more signals are needed because the work happens in many steps. These signals record the intermediate reasoning steps, the calls to external tools and APIs, and the retrieval-augmented generation chains.

Techniques and methods

Techniques associated with AI observability include:

End to end tracing of requests as they move through pipelines, chains, or agent steps, so that each stage of the execution becomes visible.

Structured logging of events, including model identifiers, timestamps, and request and

response pairs.

Anomaly detection over the telemetry, in order to surface unexpected behavior.

Synthetic monitoring, in which predefined test prompts are sent repeatedly, for example after a model or prompt update, to detect regressions.

Automated evaluation, including the use of one language model to assess the output of another, an approach often called "LLM-as-a-judge", for qualities that are difficult to measure with conventional code.

Human-in-the-loop review at important checkpoints.

Open source tools and standards

Because AI observability is a new field, several open standards and open source tools have been created to avoid tying telemetry to a single vendor. The most notable effort is the set of semantic conventions for generative AI defined by OpenTelemetry, a Cloud Native Computing Foundation project. These conventions describe a common way to record model calls, token usage, tool and agent calls, and evaluation scores, so that telemetry produced by one tool can be read by another. As of 2026, these conventions are still under active development and are not yet marked as stable.

Open source instrumentation libraries can generate telemetry that follows these conventions automatically for common model providers and frameworks. Separately, several open source projects focus on tracing and evaluating machine learning and LLM applications, and some of them can export or import data in the OpenTelemetry format, which allows them to work together with general purpose observability backends instead of only their own dashboard.

The following table lists representative open source tools used in AI observability pipelines. OpenTelemetry and MLflow are oriented specifically toward machine learning workloads; Prometheus and Grafana are general-purpose tools that commonly serve as the metrics and visualization backends to which AI-specific components export data.

Proprietary platforms

Several commercial platforms offer AI observability as a managed service, integrating tracing, evaluation, and monitoring into a single product. The following table lists representative vendors; it is not exhaustive.

Agent observability

The observability of autonomous agents is often treated as a distinct sub-area, because agents navigate multi-step workflows, make autonomous decisions, and invoke external tools, producing execution paths that are more complex than those of single-prompt model calls. Practitioners distinguish evaluation, which tests whether an agent can perform a task before or during deployment, from observability, which monitors whether it is performing correctly once deployed.

Tracing an agent execution is typically structured as a hierarchy of spans, in which each model invocation, tool call, memory access, or delegation to a sub-agent constitutes a child span of the enclosing request. This structure allows the latency, cost, and error state of each individual step to be examined in isolation, and makes it possible to identify which step in a multi-step pipeline produced an incorrect or unsafe intermediate result. Capturing the intermediate reasoning that an agent produces before committing to an action is also treated as a distinct telemetry signal, because it allows operators to assess not only what the system did but why it chose to do it.

Relation to MLOps

AI observability is treated in the literature as the runtime monitoring component of MLOps (machine learning operations), the set of practices that combines machine learning, software engineering, and DevOps disciplines to deploy and maintain ML systems in production.

Monitoring is identified as one of the core functional components of the MLOps lifecycle, alongside data management, model training, and continuous delivery.

The need for dedicated production monitoring of ML systems was identified as a source of hidden technical debt by Sculley et al. (2015), who showed that deployed ML pipelines incur ongoing maintenance costs from data dependencies that shift without notice, from feedback loops in which model outputs influence future inputs, and from undeclared consumers of model predictions, all of which make behavioral change difficult to detect with conventional software testing alone. AI observability provides the runtime telemetry required to surface such changes as they occur in production.

MLOps describes the full lifecycle of an ML system from data preparation through training, deployment, and maintenance; AI observability refers specifically to the instrumentation and analysis that make the system's production behavior transparent. In this framing, AI observability is the operational monitoring layer within MLOps, with the additional requirement, for systems based on large language models or other generative models, that it track behavioral and semantic signals beyond the infrastructure metrics typical of conventional software systems.

Relation to explainability

AI observability is related to but distinct from explainable artificial intelligence (XAI), the field that studies how model decisions can be made interpretable to users and stakeholders. The two fields address different questions: observability asks whether a system in production can be shown to be behaving correctly from external signals, while explainability asks why a particular output was produced given the model's internal parameters.

XAI methods, such as feature attribution techniques, counterfactual explanations, and surrogate models, typically operate by examining model internals or constructing approximations of individual predictions. Observability, by contrast, collects external signals

including inputs, outputs, latency, and drift measures, without requiring access to the model's weights or architecture. This makes observability applicable to systems for which internal state is unavailable, including models accessed through commercial APIs.

The two approaches are considered complementary. Feature attribution scores or other XAI outputs can be treated as additional observability signals and stored alongside conventional telemetry, making it possible to monitor shifts in the reasoning patterns behind model outputs over time. Conversely, observability data that reveals a change in a model's output distribution may prompt the application of XAI methods to investigate the underlying cause.

Regulatory and compliance context

National and regional regulatory instruments have begun to codify production-monitoring obligations that align with what AI observability tools are built to do.

The Artificial Intelligence Act, Regulation (EU) 2024/1689 adopted by the European Union in 2024, lays down a risk-based framework for AI systems. Article 12 requires that high-risk AI systems technically allow for the automatic recording of events over the lifetime of the system, so that logs can be used to identify risk situations and support post-market monitoring. Article 72 requires providers of those same systems to establish and document a post-market monitoring program that actively and systematically collects, documents, and analyzes relevant performance data throughout the system's lifetime. For covered systems, the two provisions together effectively mandate the instrumentation and logging infrastructure that AI observability is designed to provide.

The National Institute of Standards and Technology (NIST) published the Artificial Intelligence Risk Management Framework (AI RMF 1.0) in 2023 as a voluntary guide for managing AI risk across the system lifecycle. The framework's MEASURE function calls for quantitative, qualitative, or mixed-method tools to analyze and monitor AI risk on an ongoing basis. Subcategory MEASURE 2.4 states that the functionality and behavior of the AI system and its

components should be monitored when in production, which is the core purpose of AI observability telemetry pipelines.

Security, privacy and governance

Because AI observability may involve capturing prompts and responses, which can contain personally identifiable information (PII), sensitive data is typically redacted before storage to reduce privacy and compliance risk. Observability data is also used to maintain audit trails, enforce access control, and detect anomalous or malicious usage such as excessive querying, prompt injection attempts, or attempted data exfiltration. These functions connect AI observability to the broader fields of AI safety and AI governance.

Challenges and limitations

Several challenges have been noted. Machine learning models are often described as "black boxes" whose internal decision-making is not directly interpretable, complicating the inference of internal state from outputs. Defining objective measures of output quality is difficult, and a system's infrastructure-level health may be uncorrelated with the correctness of its outputs. Capturing detailed telemetry, particularly the full content of prompts and responses, also carries storage costs and privacy implications.

See also

Concept drift

Explainable artificial intelligence

MLOps

Observability

Observability (software)

References

External links

Robertson, Derek; Kosinski, Matthew (30 December 2025). "What is AI observability?". IBM Think. Retrieved 2026-07-03.

"OpenTelemetry for Generative AI". OpenTelemetry. 5 December 2024. Retrieved 2026-07-03.

"AI Risk Management Framework". National Institute of Standards and Technology. 12 July 2021. Retrieved 2026-07-03.

"Regulation (EU) 2024/1689 (Artificial Intelligence Act)". EUR-Lex. Retrieved 2026-07-03.